

Enhanced LDPC Coding for 3-D TLC NAND Flash Memory: Leveraging RBER Difference From Intralayer Variation

Lanlan Cui^{ID}, *Member, IEEE*, Fei Wu^{ID}, *Member, IEEE*, Meng Zhang^{ID}, Zhazhan Zhao, Xuan Zhang, Kun Jiang, and Changsheng Xie^{ID}, *Member, IEEE*

Abstract—NAND flash memory employs high code rate low-density parity-check (LDPC) codes to reduce the amount of redundant data that must be added. When the code rate is high, although the redundancy space is small, the error correction capability is inferior to medium or low code rate LDPC. Raw bit error rate (RBER) varies among the storage layers for 3-D triple-level cell (TLC) NAND flash memory, which increases the frequency of read retry operations. Repeatedly initiating read retry seriously increases the decoding latency and decreases the performance of the 3-D TLC NAND flash memory. To alleviate this problem, this article proposes IntraLayer Variation aware LDPC coding, called LVLDP. The proposed LVLDP scheme establishes the correlation between interlayer interference and RBER based on a neural network model. By analyzing and predicting RBER through the neural network model, we are able to categorize RBER into distinct levels. Then, we then select LDPC codes with appropriate error correction capabilities to decode data with varying levels of RBER. Through this scheme, we do not need to start read retry when $RBER < 1.56 \times 10^{-2}$. The iteration number is reduced by 67% in total. This scheme only causes 1.15% space overhead, which is negligible. For the stage with high RBER, the number of iterations of LVLDP is still large, and the extended LVLDP scheme (eLVLDP) is further proposed to reduce the use of high code rate and reduce

the number of iterations by 19.1%, expanding the correctable RBER threshold to 2.68×10^{-2} .

Index Terms—3-D triple-level cell (TLC) NAND flash, code rate, low-density parity-check (LDPC), raw bit error rate (RBER).

I. INTRODUCTION

NAND flash has been widely used in various consumer products, such as mobile phones, cameras, and computers to store data [1], [2], [3]. With constant technology scaling, more and more bits are stored in one NAND flash cell, resulting in a sharp increase in raw bit error rate (RBER). Error correction code (ECC) can effectively improve the data reliability, thus extending the NAND flash lifetime. Low-density parity-check (LDPC) codes [4], [5] as the popular ECC, are commonly used in 3-D triple-level cell (TLC) NAND flash due to their better performance than the traditional Bose–Chaudhuri–Hocquenghem (BCH) codes.

For 3-D TLC NAND flash memory, the generation of interlayer interference makes RBER more elusive [6]. Unlike errors caused by program/erase (P/E) cycles and retention time, interlayer interference does not have strong regularity but appears randomly under different circumstances. The code rate selection for LDPC should be based on RBER. Interlayer interference introduces significant complexity into the RBER, exhibiting severe fluctuation across various storage layers, characterized by notable disparities in RBER. Consequently, the application of a uniform code rate LDPC decoding scheme poses a challenge in effectively accommodating these variations in RBER. It is important to choose the code rate that matches RBER. For LDPC, error correction capability and redundant space are two contradictory factors that are difficult to satisfy simultaneously. In general, to store more effective information in TLC NAND flash, the code rate of LDPC is high, such as 0.9. The management module of the TLC NAND flash memory chip implements the read retry scheme [7]. When the RBER is high, the LDPC code may not be able to fully correct errors, triggering the activation of the read retry scheme. This results in re-reading and decoding of data stored in TLC NAND flash memory. If the decoding is still incorrect, this process will continue until either correct decoding is achieved or the maximum number of read retries is reached, at which point the retry ends. Although the redundancy space of

Received 20 January 2025; revised 7 May 2025 and 26 July 2025; accepted 9 August 2025. Date of publication 14 August 2025; date of current version 23 February 2026. This work was supported in part by the National Natural Science Foundation of China under Grant UA22A2071 and Grant 62372197; in part by the National Key Research and Development Program of China under Grant 2022YFB2804303; in part by the Shandong Provincial Natural Science Foundation under Grant ZR2024LZH004; in part by the Joint Fund for Intelligent Computing of Shandong Natural Science Foundation under Grant ZR2023LZH007; and in part by the Natural Science Basic Research Program of Shaanxi Province under Grant 2025JC-YBQN-835. This article was presented in part at the IEEE International Conference on Computer Design (ICCD) 2024 [DOI: 10.1109/ICCD63220.2024.00080]. This article was recommended by Associate Editor A. Aminifar. (*Corresponding author: Fei Wu.*)

Lanlan Cui is with the School of Computer Science and Engineering and the Shaanxi Key Laboratory for Network Computing and Security Technology, Xi'an University of Technology, Xi'an 710048, China, and also with the China-ASEAN Information Harbor Company Ltd., Nanning 530200, China (e-mail: cuilanlan@xaut.edu.cn).

Fei Wu and Changsheng Xie are with the Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: wufei@hust.edu.cn; cs_xie@hust.edu.cn).

Meng Zhang is with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: zgmeng@hust.edu.cn).

Zhazhan Zhao, Xuan Zhang, and Kun Jiang are with the School of Computer Science and Engineering and the Shaanxi Key Laboratory for Network Computing and Security Technology, Xi'an University of Technology, Xi'an 710048, China (e-mail: 1983440957@qq.com; lhzhangxuan@xaut.edu.cn; jk_365@xaut.edu.cn).

Digital Object Identifier 10.1109/TCAD.2025.3598931

high code rate LDPC is small, the error correction capability of LDPC is not enough, which leads to an increase in the number of using read retry when facing high RBER. Once read retry is activated multiple times, the decoding latency is seriously increased, and affects the read performance of the TLC NAND flash memory.

To solve the above issue, much research was proposed. Some researches were based on the design of LDPC codes [8], [9]. Another type was about decoding algorithms. LAE-FTL was proposed in [10], which adopts a lifetime-adaptive ECC scheme, using weak ECC in the early stage and strong ECC in the later stage to improve the performance and ensure storage reliability. Reference [11] used the clustering algorithm to classify the bit error rates of different pages on the same word line of the 3-D TLC NAND flash memory into different categories. When performing error correction, LDPC codes of different code rates can be selected to save storage overhead. Wu et al. [12] developed the LDPC decoding strategy to optimize the log-likelihood ratio (LLR) information based on the error modes of 3-D TLC NAND flash. However, these researches do not consider the differences between different layers of 3-D TLC NAND flash memory.

Motivated by the above analysis, this article proposes an IntraLayer Variation aware LDPC coding scheme for 3-D TLC NAND flash, called LVLDP. First, we use back propagation (BP) neural network to model the relationship between program/erase (P/E) cycles, retention time, interlayer interference, and RBER. Based on this model RBER is analyzed and predicted under different program/erase cycles (P/E cycles) and retention time, and RBER can be divided into different levels. Then, we select the LDPC code with various error correction abilities to match different levels of RBER. We give two types of code rate choices, 5 code rate conversions and 3 code rate conversions, the latter of which can cope with high RBER. Through LVLDP, we can not start read retry when $RBER < 1.56 \times 10^{-2}$, and when $RBER < 2.51 \times 10^{-2}$, we reduce the number of read retry from greater than nine to three. And the decoding latency is reduced. The enhanced extended LVLDP scheme (eLVLDP) extends this capability, suppressing read retry initiation at higher error thresholds up to $RBER < 2.68 \times 10^{-2}$.

The remainder of this article is organized as follows. In Section II, we introduce the background and motivation of our article. Section III proposes the LVLDP. First, the BP network model of RBER is introduced, and then the code rate is selected based on RBER. Section IV evaluates the effectiveness of the proposed scheme. Conclusions are drawn in Section V.

II. BACKGROUND AND MOTIVATION

A. 3-D NAND Flash

1) *3-D NAND Flash Structure*: Each TLC NAND flash cell represents three bits of information, namely, the least significant Bit (LSB), the central significant bit (CSB), and the most significant Bit (MSB). 3-D NAND flash stacks multiple layers vertically to overcome the capacity limitations of 2-D NAND flash, as shown in [13]. This architecture allows for higher density without sacrificing data integrity, thus enabling

more storage capacity. While the 3-D stacking technology significantly enhances the storage density of NAND flash memory, it also poses several challenges. First, it introduces greater complexity to the NAND flash memory structure and increases channel noise diversity, thereby posing a heightened threat to data reliability [14]. Second, due to the unique production process characteristics of 3-D NAND flash memory, there exist variations in the physical structure of storage units across different layers. These variations manifest as significant differences in the RBER among layers, which is a key metric for evaluating data reliability in NAND flash memories.

2) *3-D TLC NAND Flash Errors*: Despite the increased storage capacity, data errors are becoming more prevalent. As the threshold voltage distribution of each state shifts and widens, raw bit errors arise during read operations. The article primarily investigates three prevalent factors that contribute to data storage errors in 3-D TLC NAND flash, namely, P/E cycle, retention time, and interlayer interference [15], [16].

The increasing number of P/E cycles causes electrons to be trapped in the tunnel oxide. The accumulation of trapped electrons results in wear-out, which in turn leads to errors in the data stored in 3-D TLC NAND flash. Retention errors are another major source in 3-D TLC NAND flash, resulting from charge leakage over time after a 3-D TLC NAND flash cell is programmed. The last factor is a specific characteristic called interlayer interference, which is unique to the 3-D stacking structure. As its name suggests, this type of error is caused by the interference between adjacent layers during the programming process.

The above factors result in variations in the threshold voltage of the 3-D TLC NAND flash cell, which represents the data storage value. If the pattern of variation in errors cannot be tracked, numerous data errors will occur during reading and programming operations.

B. LDPC Decoding

LDPC decoding includes hard-decision decoding and soft-decision decoding. The two ways complement each other to meet the error correction needs of NAND flash [17]. To ensure the data reliability of 3-D TLC NAND flash, the read retry scheme is implemented. When a data error occurs during the read process, the hard-decision decoding procedure is initiated first. Hard-decision decoding involves applying a single read reference voltage during data retrieval. This approach is straightforward to implement as it relies solely on symbol features. However, its error correction performance is relatively limited. Another advantage of hard-decision decoding is that the decoding convergence speed is fast and the decoding latency is small. When RBER is high enough to exceed the ability of hard-decision decoding, the soft-decision decoding starts. If the hard-decision decoding succeeds, the decoding process ends, and the result is outputted. Conversely, if the error persists after hard-decision decoding, the soft-decision decoding method is activated. Soft-decision is realized by applying multiple read reference voltages to read data and decoding multiple times. Soft-decision decoding has a good ability to correct errors, but adopting soft-decision decoding results in significant read latency. If hard-decision decoding

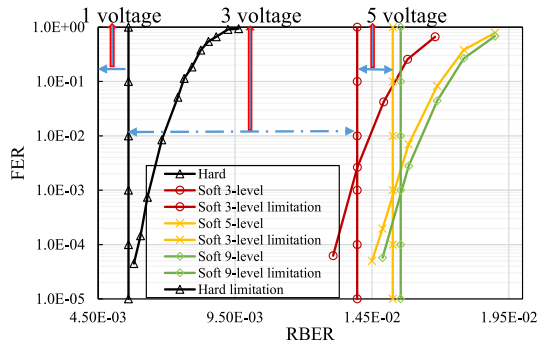


Fig. 1. Error correction capability level of different decoding schemes.

can be improved to enhance its error correction capability, the threshold for initiating soft-decision decoding can be raised, thereby postponing the activation of soft-decision decoding. Therefore, the overall latency is reduced directly.

C. Motivation

In 3-D TLC NAND flash memory, in order to reduce the use of redundant space, high code rate LDPC, such as around 0.9 is often used. However, this type of LDPC cannot cope with high RBER. As shown in Fig. 1, when $RBER > 5.61 \times 10^{-3}$, hard-decision cannot correct the errors, soft-decision is started. If $RBER < 1.4 \times 10^{-2}$, three reference voltages are enough to ensure excellent error correction capability, accompanied by greater decoding latency than hard-decision decoding. Once $RBER > 1.4 \times 10^{-2}$, more voltages are applied, resulting in large read latency. After the RBER exceeds $> 1.56 \times 10^{-2}$, the soft decision is powerless, and many unsustainable errors appear. When the voltage continues to increase, read retry is continuously activated during decoding. This process causes serious decoding latency. If the number of starting read retry can be reduced while ensuring good error correction performance, the read performance of 3-D TLC NAND flash memory can be greatly improved.

Whether it is hard-decision decoding or soft-decision decoding, the traditional way is to use a high code rate of LDPC for error correction, without considering the variation of different layers. Under these circumstances, although there is a lot of effective information to store, the error correction capability of LDPC decoding is limited. However, when the RBER is relatively high, correcting errors with LDPC decoding becomes challenging, requiring numerous iterations and resulting in significant decoding latency. Therefore, to explore the impact of the layer on RBER, we perform the test and analysis of RBER. The data comes from the test of the 3-D floating gate (FG) TLC under different P/E cycles, retention time and NAND flash layer. We test the RBER of LSB, CSB and MSB for the various layers under different P/E cycles and retention time. The hardware platform of the 3-D TLC NAND flash sorting test board is connected to the host through a PCIe interface to perform the tests. The test time can be reduced by baking the chip where the data is written at a high temperature according to the Arrhenius Law [18]. By writing random data and reading data under various NAND flash states, and statistics data errors, RBER is calculated.

For 3-D TLC NAND flash memory, the RBER between different layers varies greatly, as shown in Fig. 2. This is from our analysis of the test data of the real platform. The P/E cycles include 1000, 3000, and 5000. The retention periods are 3-month and 6-month. We use (PE, RT) to describe the state of the data, referring to the P/E cycle and retention time. The horizontal axis represents the storage layer of 3-D TLC ANAND flash. The left vertical axis represents iteration number required for decoding, and the right is the RBER. For a given layer, errors induced by P/E cycles tend to increase with an increasing number of P/E cycles. Similarly, retention time also contributes to the accumulation of errors. But for different layers, there is no obvious rule for the occurrence of errors, and there are great differences between adjacent layers. Specifically, for MSB with (PE, RT) = (5, 6) and CSB with (PE, RT) = (5, 6), the number of iterations used for decoding fluctuates greatly among different layers. In addition, for CSB with (PE, RT) = (5, 6) with the high RBER, in order to correct as many errors as possible, the number of iterations approaches 30 for almost all layers.

The instability of the number of iterations leads to inconsistencies in decoding latency, which affects the normal use of 3-D TLC NAND flash memory. For the other P/E cycle and retention time, the difference in RBER among different layers is not particularly obvious. The inconsistency of the RBER differences among layers in different flash cells makes RBER more complicated. Interlayer interference causes RBER to become complicated. Since the design of the LDPC decode is dependent on RBER, it is important to establish the model to describe internal relationship between NAND flash layer and RBER. This provides a theoretical basis for choosing the appropriate code rate under different flash cell states. Combined with the above analysis, we need to establish a model of RBER and layer number and design an LDPC strategy to reduce the start of read retry.

III. PROPOSED SCHEME

A. Overview

The design of this article mainly includes three modules, test data of 3-D TLC NAND flash, RBER model establishment and RBER prediction, and decoding by using code rate conversion scheme, corresponding to module 1, 2, and 3 in Fig. 3. For module one, we carry out the test on the real FPGA platform to obtain data. Module two performs a two-part operation. First of all, we establish a BP neural network model of RBER based on the P/E cycle, retention time, and layer number. Then, after getting the actual data, we predict the RBER according to the external information of the data received by the decoder, that's P/E cycle and retention time. For module three, according to the result of module two, we choose code rate of LDPC that matches RBER based on the neural network model.

B. Proposed Scheme

1) *Test Model*: As described in the motivation section, we test RBER under different P/E cycles, retention time, and layer number for three types of pages. The retention time can be reduced by baking the chip where the data is written at a

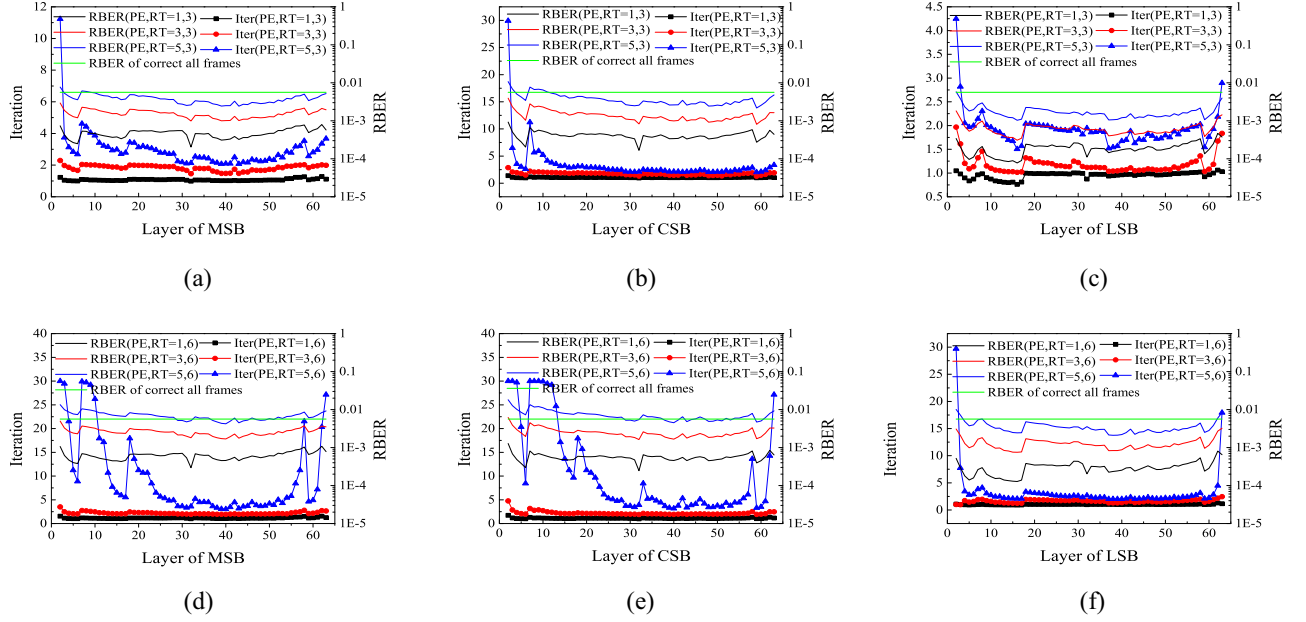


Fig. 2. Iteration number under different P/E cycles and retention time for three types of pages. (a) UP,RT = 3. (b) CSB,RT = 3. (c) LSB,RT = 3. (d) MSB,RT = 6. (e) CSB,RT = 6. (f) LSB,RT = 6.

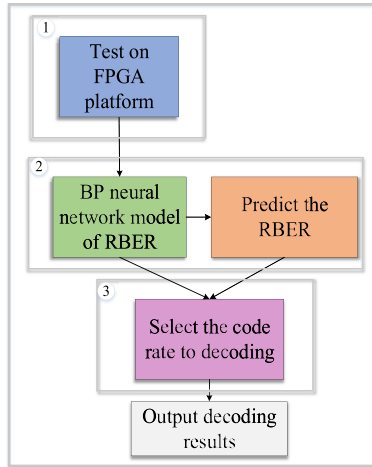


Fig. 3. Architectural overview of LVLDP.

high temperature. The test is based on a real FPGA platform. RBER is obtained by randomly writing data and reading the disturbed data.

In addition, in order to select an appropriate code rate conversion scheme, we need to know the error correction performance of LDPC with different code rates. Here, the error correction capability of LDPC code refers to the worst RBER that errors of all frames can be corrected. We use 20000 frames to test the correctable RBER upper limit for different code rates. Use the LDPC with a special code rate for decoding, as the RBER decreases, the decoding error rate becomes lower and lower. When no errors occur in all frames, the RBER is the error correction limit we are looking for. This means that when RBER is less than this value, all errors can be corrected.

We show the error correction capabilities (written as “L”) of different code rates in Fig 4. We take the position of FER

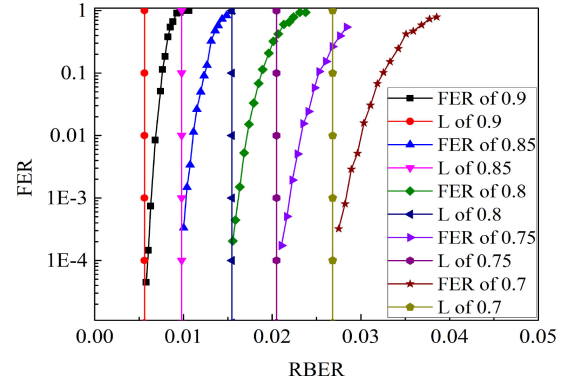


Fig. 4. Error correction of different code rates.

equal to 1×10^{-4} as the division standard to measure that the data is no longer wrong after decoding, that is when the decoding result is less than 1×10^{-4} , we believe that the data is no wrong. It can be seen that the RBER is divided into six parts, and each part corresponds to a code rate. The five code rates are 0.9, 0.85, 0.8, 0.75, 0.7, from left to right. These code rates are selected because they can correct most of the errors that occur in the current test, and it is meaningless to further reduce the code rate. The baseline code rate of 0.9 aligns with industry-standard flash memory configurations, while the four additional rates are derived via progressive shortening operation [19]. Notably, reduced code rates exhibit dual performance advantages: improved post-decoding FER alongside expanded tolerance for higher RBER. In Fig 4. it is observed that when the LDPC code rate is set to 0.85, the limit of RBER that can be corrected is approximately 0.01. That is to say, for RBER values below 0.01, the LDPC code with 0.85 code rate is capable of correcting all errors incurred.

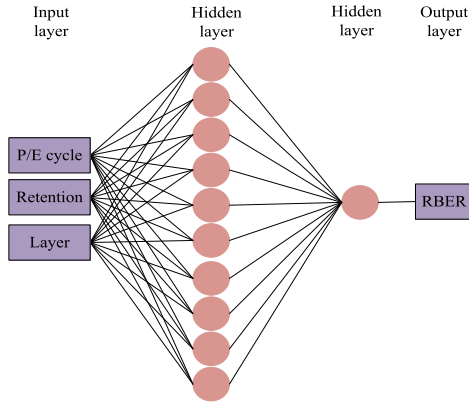


Fig. 5. BP neural network for fitting RBER.

2) *RBER Model Based on BP Neural Network*: When building the model, we initially consider using the traditional linear regression method to predict RBER. We treat P/E cycles, data retention time, and the number of layers as three independent variables, while RBER is the dependent variable. Subsequently, we conduct multiple linear regression analyses on the data from LSB, CSB, and MSB pages, with each dataset containing 550 sample points. The fitting results show that the data roughly follows a linear relationship, but the accuracy of the fit is not high. Specifically, the fitting results for the three page types, as indicated by the R^2 values, are 0.66, 0.72, and 0.77, respectively. To improve the accuracy of the predictions, we then attempt to fit RBER using machine learning algorithms.

Because the RBERs of LSB, CSB, and MSB vary, we build BP neural network models for the three types of TLC pages respectively. For each page, the idea of building a model is the same. The network architecture comprises four distinct layers, as depicted in Fig. 5. The initial layer, designated as the input layer, transmits parameters, such as P/E cycles, retention time, and layer number to the subsequent hidden layer. This first hidden layer performs computational processing to generate messages, which are then forwarded to the second hidden layer. Following the updating of calculation results within the second hidden layer, the final layer, referred to as the output layer, integrates the outcomes to yield RBER.

The BP algorithm directly employs the prediction error of the output layer to inversely propagate the error of preceding layers, progressing from the output to the input layer in a reverse order. This backward propagation of errors serves to adjust the interlayer weights. In our model, the discrepancy between the predicted RBER and the RBER in the original data is calculated and propagated back through the layers. Subsequently, the weights at each layer are adjusted to progressively minimize the deviation between the predicted RBER and the original data. The training function utilized for BP is the momentum gradient descent algorithm.

We utilized a sample set comprising 70% of the data, encompassing P/E cycles of 1000, 3000, and 5000 as well as retention periods of 3 months and 6 months. The remaining 30% of the data constituted the test set. Employing this BP

TABLE I
ERROR CORRECTION CAPABILITIES FOR DIFFERENT CODE RATE

Code rate	0.9	0.85	0.8
L	5.61×10^{-3}	9.83×10^{-3}	1.55×10^{-2}
Code rate	0.75	0.7	
L	2.05×10^{-2}	2.68×10^{-2}	

neural network, we achieved high accuracy in predicting the performance of three types of pages. We assessed the accuracy of our predictions using the R^2 metric. As illustrated in Fig. 6, the prediction errors for the three page types were minimal, with R^2 values of 0.90, 0.97, and 0.93, respectively. On average, our prediction accuracy reached 93%. The complexity of RBER has posed challenges in devising effective decoding strategies. The model we have developed can accurately capture the relationship between various influencing factors and RBER. Accurate prediction of RBER is crucial for the smooth progression of subsequent steps. Given the high accuracy achieved by our BP neural network, we can select the most suitable LDPC code for the given application. RBER prediction is inherently challenging. This work provides only a coarse estimation, as our focus is not on prediction accuracy but on maintaining framework integrity. Advanced prediction methods may further enhance precision in practical applications.

The RBER prediction model adopts an offline execution mode and generates a dynamic code rate configuration table by integrating parameters, such as P/E cycle, data retention time, and storage layer location. The specific implementation includes two stages. In the prediction stage, an RBER model is established based on historical data to fit and predict the RBER evolution trajectory of each storage layer. In the storage phase, based on the code rate to RBER mapping relationship in Table I, only the layer configuration index for non reference code rates ($\neq 0.9$) is stored, with a storage overhead of 1 KB.

3) *Code Rate Selection*: Based on the statistical analysis and predictions derived from the network model, we can determine the comprehensive range of RBER values. Subsequently, within this range, we categorize the RBER in accordance with the error correction capabilities of LDPC codes possessing varying code rates. Consequently, in this section, we have conducted preliminary experiments to ascertain the specific RBER thresholds at which all data can be successfully corrected for each code rate. In Module 1, we have established the error correction capabilities of LDPC codes with diverse code rates. As a subsequent step, we intend to expand upon this foundation and elaborate a specific scheme, which constitutes the primary focus of Module 2.

To provide a more comprehensive understanding, we present in Table I a detailed depiction of the correlation between code rate and error correction capability of LDPC. In this context, “L” serves as an indicator for the RBER limitation. As evident from the analysis, the imbalance in RBER leads to significant variations in the number of decoding iterations. Specifically, in certain scenarios, the decoding latency is observed to be notably prolonged. To address this issue and ensure consistent

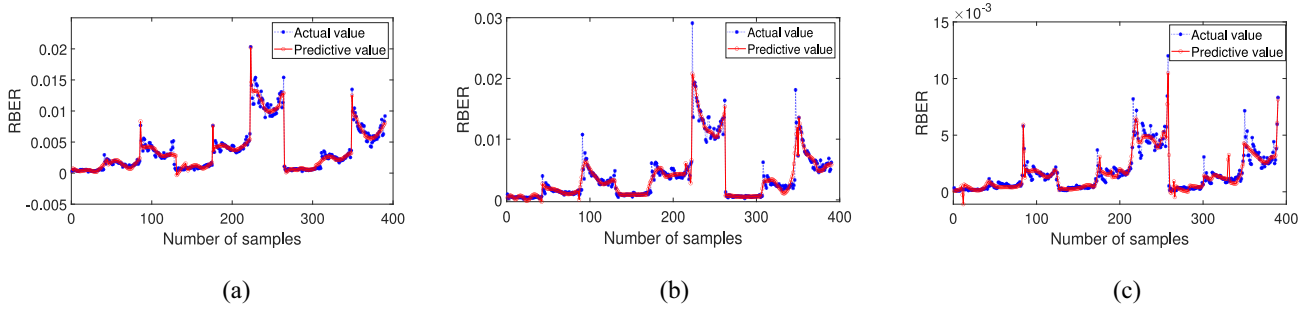


Fig. 6. Predictions of (a) MSB, (b) CSB, and (c) LSB.

decoding latency, we employ LDPC codes with low code rates in high RBER environments.

Initially, we predict the RBER utilizing the BP neural network model presented in the previous section. Subsequently, we compare the obtained RBER value with the threshold “L” specified in Table I to ascertain the appropriate code rate for error correction. Specifically, when the RBER falls below a given threshold “L,” we select a code rate that is higher than that threshold for error correction. During our testing, we observed that most RBER values were below 5.61×10^{-3} , indicating that an LDPC code with a 0.9 code rate is sufficient for error correction. However, the focus of this article’s LVLDP scheme is primarily to address cases where the RBER exceeds 5.61×10^{-3} .

As shown in Table I, each code rate corresponds to a specific RBER threshold interval. As long as the inaccurate RBER prediction value is still within the error correcting interval of the same code rate, it will not affect the system’s error correction effect. For example, when the predicted RBER is 4.2×10^{-3} and the actual value is 5.2×10^{-3} , both are within the error correcting range of 0.9 code rate, so it does not affect system performance.

To gain a more profound understanding of the application of this scheme, we offer an illustrative example. First, we ascertain the RBER based on factors, such as the P/E cycle, retention time, and layer number. Subsequently, we categorize the obtained RBER value into its corresponding range specified in Table I. Assuming the determined RBER is 1.5×10^{-2} , we select a code rate of 0.8 accordingly. Following this determination, we proceed with the standard decoding process, and once the termination condition is met, the decoding results are subsequently outputted. The algorithmic implementation of this scheme is detailed in Algorithm 1.

Ensure:

In this proposed scheme, the variation of code rates is achieved through the utilization of shorten technique, which offers significant convenience for storing a matrix to accommodate diverse code rates. The meaning of the shorten algorithm is to reduce the length of information bits in the codeword, thereby reducing the code rate, that is the length of the codeword involved in transmission is shortened accordingly. Assuming the original code length is Sn , the length of the information storage bit is Sk , and the length that shorten technique needs to handle is Sx , then code rate after shorten operation is $(Sk - Sx)/(Sn - Sx)$. Specifically, the shortening

Algorithm 1 Implementation Algorithm of Selecting Rate

Input: p_n : Received initial LLR information from the channel.

Output: y_n : Decoding result of the k^{th} iteration.

Assuming that the pre-divided RBER interval is L .

Set the error correction limitation as C , and the maximum iteration number as m .

```

1: Calculate RBER of  $p_n$ ;
2: for  $i = 1; i < n; i++$  do
3:   if  $RBER < L[i]$  then
4:      $Code\_rate = C[i]$ .
5:     Perform decoding with  $C[i]$  code rate.
6:     if  $\vec{y}_n H^T = 0$  or  $k^{th} = m$  then
7:       Decoding end. Output the decoding result  $y_n$ .
8:       break;
9:     end if
10:  end if
11: end for

```

operation involves modifying a portion of the message input to the decoder by setting it to 0 prior to decoding. This approach serves to limit the propagation of information, thus effectively reducing the code rate. By leveraging this method, we only require the storage of the H matrix corresponding to a code rate of 0.9. Notably, in comparison to a single-rate scheme, our approach does not incur any additional overhead.

The system first executes the conventional decoding process with an initial code rate of 0.9. When decoding fails, the code rate for redecoding is selected based on the RBER and activates the read retry mechanism, i.e., re-reading the data and reperforming decoding. The correctly decoded data undergoes a re-encoding process, as shown in Fig. 7. Taking 0.8 code rate as an example, this process requires two independent codeword shortening operations: the first targets the first half of the codeword, and the second targets the second half. The shortened bits are fixed to the logical value 1 and assigned the maximum LLR value under the hardware bit-width limit, serving as known messages in subsequent processing. After the two shortening operations are completed, the system automatically calculates and updates the code rate to 0.8, then performs encoding based on this code rate and stores the data in the flash memory. When the data needs to be read, even if errors occur in the data, the 0.8 code rate can provide stronger error correction capability. It is worth noting that the

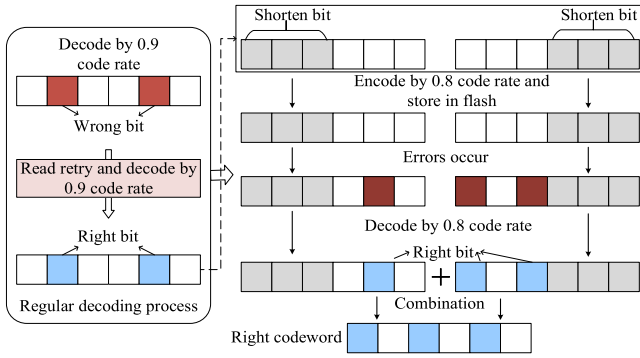


Fig. 7. Schematic diagram of reducing code rate through shortening operation.

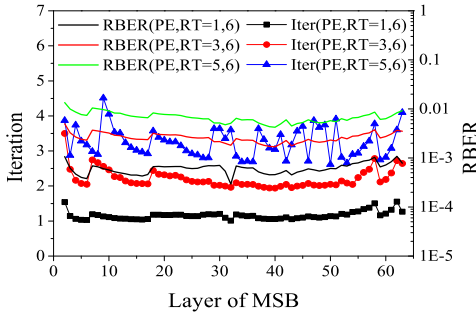


Fig. 8. Comparison of the number of voltages used.

two shortening operations can be executed in parallel through a hardware pipeline to improve processing efficiency. The decoding result of the first shortening operation is stored in the cache, while the decoding result of the second shortening operation directly covers the remaining unprocessed area.

The frequency of code rate 0.85 and 0.8 startup is 7.82% and 6.7%, respectively. The frequency of code rate 0.75 and 0.7 startup is low, at 0.55% and 0.06%, respectively. The re-encoding operation is dispersed to non real time tasks in the background, reducing the impact on flash system performance.

4) *Extended LVLDP Scheme*: As the usage of flash memory progresses, both the P/E cycles and data retention time increase, resulting in a gradual rise in the RBER. When the RBER surpasses a certain threshold, the current LVLDP becomes insufficiently effective due to the persistently high number of decoding iterations, as depicted in the accompanying Fig. 8. The data in this figure is the same as that in Fig. 2(b). To enhance clarity, we adjusted the maximum value of the data's coordinate axis, allowing for a more discernible view of the iteration number. The figure specifically illustrates the iteration number required for the MSB with $(PE, RT) = (5, 6)$ is higher than that of $(PE, RT) = (3, 6)$. This adjustment underscores the inadequacy of the current LVLDP mechanism in certain scenarios.

To address this issue and further reduce both the number of iterations and decoding latency, we introduce an enhanced version of the LVLDP mechanism, termed eLVLDP. The original LVLDP employs five distinct LDPC code rates to cater to the varying error correction requirements at different

TABLE II
RANGE OF RBER THAT CAN BE CORRECTED BY DIFFERENT CODE RATES FOR THE eLVLDP

Code rate	RBER range
0.9	$(2.68 \times 10^{-2}, 1.55 \times 10^{-2})$
0.8	$[1.55 \times 10^{-2}, 5.61 \times 10^{-2})$
0.7	$[5.61 \times 10^{-2}, -\infty)$

RBER stages. The eLVLDP extends this approach by incorporating a feature that allows for three code rate conversions. In scenarios of high RBER, such as when the MSB has undergone 5000 P/E cycles and reached a retention time of 6 months, eLVLDP employs three code rates: 0.9, 0.8, and 0.7. The specific code rates utilized across various RBER ranges are detailed in the Table II below.

Compared to the use of five code rates, the eLVLDP mechanism reduces the application of LDPC codes with rates of 0.85 and 0.75. Specifically, the RBER previously corrected by LDPC codes with a rate of 0.85 is now handled by codes with a rate of 0.8, and the RBER corrected by codes with a rate of 0.75 is now managed by codes with a rate of 0.7. While a lower code rate enhances error correction capability, it also introduces increased redundancy.

When the LDPC decoding fails to recover data, specifically when an uncorrectable error is detected, the system activates the RAID recovery mechanism. For RAID-5, after multiple failed attempts by the LDPC decoder to correct errors, the controller marks the data block as unrecoverable, issues a recovery request to the RAID management layer, and temporarily suspends user input-output operations. Next, guided by RAID-5 striping configuration, the system reads data from other blocks and the parity block within the same stripe to reconstruct corrupted content through logical operations. Finally, the rebuilt data is written to a spare storage unit with updated system mappings, after which user access is restored to complete the recovery process.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experiment Setup

In our simulation, we employed the layer min-sum algorithm and set the LDPC decoder's maximum iteration number to 30. As LDPC code, we selected the (18656, 16748) quasi-cyclic LDPC (QC-LDPC) code, renowned for its low implementation complexity [20]. Additionally, a 6 bit level quantizer is utilized. To ensure a thorough evaluation, we validated the effectiveness of the proposed scheme under various conditions, including P/E cycles of 1000, 3000, and 5000, and retention periods of 3 month and 6 month. For comparison, we utilized the traditional scheme with a fixed code rate of 0.9, while maintaining all other experimental settings identical. To demonstrate the feasibility of our proposed method, we conducted a comprehensive series of experiments.

B. Experiment Results

Utilizing LVLDP, we forecast the RBER and select LDPC codes with varying error correction capabilities for

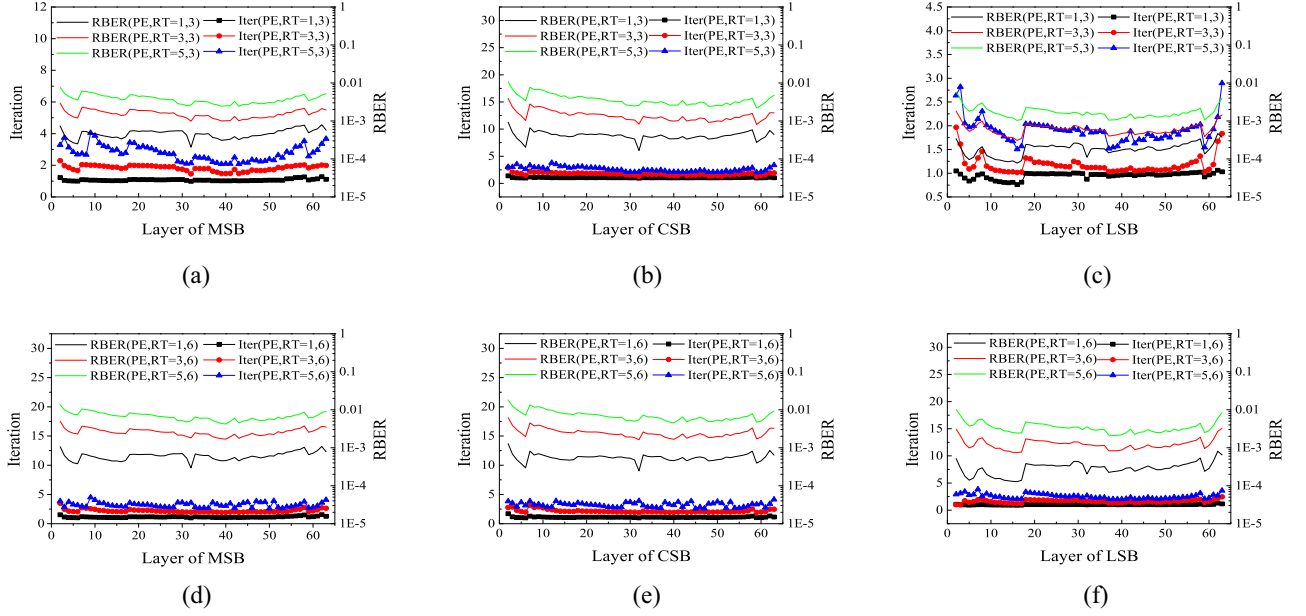


Fig. 9. Iteration number under different P/E cycles and retention time for three types of pages. (a) MSB, RT = 3. (b) CSB, RT = 3. (c) LSB, RT = 3. (d) MSB, RT = 6. (e) CSB, RT = 6. (f) LSB, RT = 6.

decoding across distinct RBER intervals. Notably, a single encoder/decoder suffices to manage all code rates, as the multicode rate capability is achieved through shortening techniques. Consequently, no additional space overhead is incurred compared to a single-rate LDPC code. Fig. 9 illustrates the iteration number required for the LSB, CSB, and MSB under the LVLDPD framework. The notation (PE, RT) represents the number of P/E cycles and retention time. The coordinate axis is the same as the previous Fig. 2.

As depicted in Fig. 9, it is evident that the iteration number for all scenarios remains below 25, exhibiting a smooth trend. Notably, the peak points in the original curves are scarcely diminished. For the MSB, regardless of the P/E cycle and retention time, the iteration number never exceeds five. Even in cases where RT is substantial, such as reaching six, the iteration number remains within five. Specifically, for the MSB under the conditions of (P/E, RT) = (5, 6), the current decoding trajectory becomes consistently smooth, eliminating sudden fluctuations, and maintaining a notably low iteration number. For the CSB, the iteration number is also less than five. In particular, for the CSB under (P/E, RT) = (5, 6), where the original curve exhibited significant oscillations across the entire coordinate axis range, our proposed scheme has mitigated this behavior, resulting in a stable decoding curve. Consistently, for the MSB, the iteration number remains low and displays a smooth trend, never exceeding five iterations in total. Generally speaking, following the adoption of LVLDPD, the curves tend to converge steadily toward a straight-line behavior, with the amplitude constrained to below five iterations.

Furthermore, for the CSB under the conditions of (P/E, RT) = (5, 6), the original iteration number approached the maximum value of 30. However, with the application of LVLDPD, this iteration number is significantly reduced to

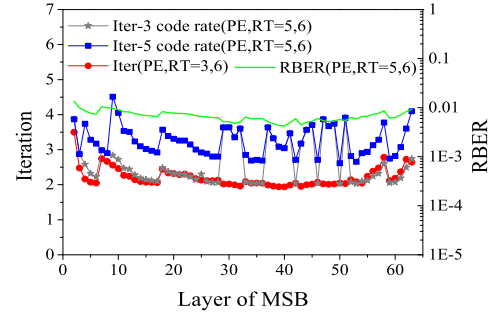


Fig. 10. Comparison of the iteration number used.

less than five. Beyond these pronounced observations, the other curves also exhibit a consistently lower iteration number compared to their original counterparts, thereby indicating the effectiveness of this strategy. The number of iterations number in LDPC decoding is closely related to decoding latency. When the number of iterations decreases, the decoding latency also decreases accordingly.

We conduct experiments utilizing the eLVLDPD method with three code rates and record the number of iterations required by the eLVLDPD mechanism. The experimental outcomes, illustrated in Fig. 10, demonstrate that eLVLDPD consistently reduces the number of iterations in comparison to the standard LVLDPD. Our analysis reveals that eLVLDPD achieves a reduction in the number of iterations by 19.1% when compared to LVLDPD, which employs five code rates. (PE, RT) = (3, 6) is used as a reference, and its iteration number is the lowest. For most layers, the iteration number of eLVLDPD is comparable to that of (PE, RT) = (3, 6). This significant decrease in iterations underscores the enhanced efficiency of eLVLDPD, indicating that it not only boasts a

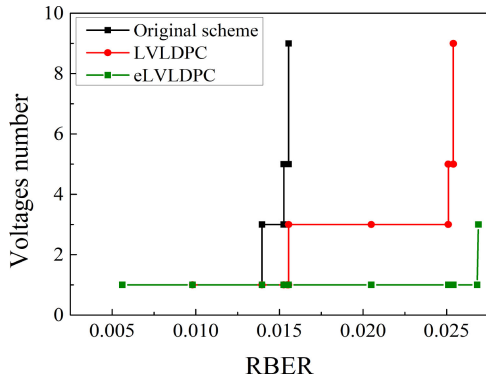


Fig. 11. Comparison of the number of voltages used.

lower decoding latency but also exhibits superior performance relative to the conventional LVDPC mechanism.

Fig. 11 presents a comparative analysis of the utilization of read retry. We employ LDPC with a single code rate as the baseline, which is also the prevailing approach. Literature [21] adopts the conventional method, named original scheme in the figure. It uses hard-decision decoding when the RBER is low. When the RBER increases and the hard-decision decoding cannot correct the error, soft-decision is initiated. As RBER continues to increase, the number that data is re-read and redecoded continues to increase. As observed in the figure, when the RBER is less than 1.4×10^{-2} , this three schemes exhibit similar performance. However, as the RBER increases, the number of voltages required for both schemes escalates, yet the original scheme experiences a more rapid increase. Specifically, after the RBER surpasses 1.56×10^{-2} , the original scheme necessitates the utilization of more than nine voltages. This study does not consider higher voltage counts due to the significant implementation latency involved. In contrast, the LVDPC scheme requires only three voltage levels for error correction, even under prolonged conditions where the RBER exceeds 1.56×10^{-2} . This approach reduces the number of initial read retry operations from over nine to three when the RBER is below 2.51×10^{-2} , achieving a 67% reduction in voltage usage across the entire operational range. Furthermore, as illustrated in the Fig. 11, LVDPC extends the correctable RBER range for hard decision decoding from 1.4×10^{-2} to 1.56×10^{-2} , thereby enhancing the error correction capability and effectively prolonging the lifetime of 3-D TLC NAND flash memory. The enhanced version, eLVDPC achieves optimal performance by activating the three-voltage mode only when RBER exceeds 2.68×10^{-2} , thereby raising the correctable RBER threshold from 1.56×10^{-2} to 2.68×10^{-2} .

Table III provides a comparison of decoding latency for MSB, LSB, and CSB under different P/E cycles and retention time. The decoding latency of LVDPC at (PE, RT) = (1,3) is used as a reference standard to calculate the relative decoding latency in literature [21]. Assuming that the decoding latency of LVDPC is defined as 1, the decoding latency is derived based on the iteration number of the two methods. Compared to [21], LVDPC has a decrease in decoding latency. In addition, we have calculated the proportion of decoding latency reduction by LVDPC for each type of page, which are 0.44%,

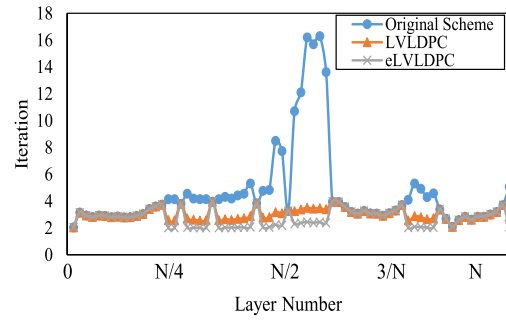


Fig. 12. Decoding results of three mechanisms for QLC flash.

TABLE III
COMPARISON OF DECODING LATENCY

Page type	MSB					
PE	1	3	5	1	3	5
RT	3	3	3	6	6	6
Original scheme	1.98	2.70	3.77	2.06	3.05	10.20
LVDPC	1.02	1.75	2.46	1.11	2.10	3.09
Page type	LSB					
PE	1	3	5	1	3	5
RT	3	3	3	6	6	6
Original scheme	1.86	2.07	2.79	1.92	2.50	4.13
LVDPC	0.91	1.12	1.81	0.96	1.55	2.42
Page type	CSB					
PE	1	3	5	1	3	5
RT	3	3	3	6	6	6
Original scheme	1.95	2.61	4.17	2.02	3.05	11.17
LVDPC	1.00	1.66	2.05	1.07	2.06	3.10

0.43%, and 0.48%, respectively. When (PE, RT) = (5,6), the threshold condition for triggering eLVDPC is met, yielding an average iteration number of 2.64, which is lower than LVDPC, which is 3.09. For other parameter combinations, LVDPC alone provides adequate correction capability, and activating eLVDPC yields identical results. Thus, redundant eLVDPC data are excluded from the Table III.

To validate the applicability of the proposed scheme for quad-level cell (QLC) flash memory, Fig. 12 compares the iteration number among the original decoding scheme, LVDPC, and eLVDPC after 1000 read cycles. The x-axis represents different layers (with total levels set to N), where the RBER data for each QLC level is sourced from [22]. As shown in the figure, the original scheme exhibits higher iteration number with significant layer-wise fluctuations. While LVDPC effectively reduces both average iteration number and interlayer fluctuations, eLVDPC demonstrates further optimization improvements. Quantitative analysis reveals that compared to the original scheme, LVDPC and eLVDPC achieve 19.7% and 24.7% reductions in average iteration number, respectively.

C. Overhead Analysis

When the RBER is elevated, LDPC decoding with matching error correction capability generates redundant information, thereby occupying valuable information space. The evaluation of this overhead comprises two distinct components. First, we estimate the RBER that is uncorrectable using traditional methods. Subsequently, we compute the storage space sacrificed as a result of this proportion. The total overhead is determined by multiplying the percentage of wasted space for each code rate by the percentage of code rate utilization across all decoding instances. Through meticulous calculations, the increased overhead for LVLDP is calculated as follows. $(0.9 - 0.85) \times 0.078 + (0.9 - 0.8) \times 0.067 + (0.9 - 0.75) \times 0.0054 + (0.9 - 0.7) \times 0.0006 = 1.15\%$, which is negligible. Similarly, the increased overhead for eLVLDP is computed as $(0.9 - 0.8) \times 0.145 + (0.9 - 0.7) \times 0.059 = 1.57\%$, also a value that can be disregarded.

D. Discussion

Compared with the 0.9 code rate LDPC solution, our LVLDP and eLVLDP are implemented based on the H matrix of 0.9 code rate LDPC code, so the storage overhead is the same. However, the throughput of the decoder is declined. Given the slight reduction in effective data storage space caused by the lower code rate, subsequent data compression techniques may be considered to mitigate the impact of this reduction.

V. CONCLUSION

In 3-D TLC NAND flash memory, RBER becomes more complicated due to interlayer interference. This article proposes an IntraLayer Variation aware multi code rate LDPC scheme for 3-D TLC NAND flash. In order to more accurately describe the relationship between layer and RBER under different P/E cycles and retention time, we established a BP neural network model. Through this model, we can predict RBER. Then, according to the correction ability of the LDPC code, the LDPC with stronger error correction ability is used when the RBER is high. Simulation shows that LVLDP can reduce the inconsistency of the number of decoding iteration among different layers, and reduce the overall number of iterations by 67% with 1.15% overhead. ELVLDP reduces 19.1% iteration number compared with LVLDP.

REFERENCES

- [1] L. Cui, M. Zhang, and F. Wu, "LVLDP: Intra-layer variation aware LDPC coding for 3D TLC NAND flash memory," in *Proc. IEEE 42nd Int. Conf. Comput. Design (ICCD)*, 2024, pp. 483–486.
- [2] Y. Du, Y. Gao, S. Huang, and Q. Li, "LDPC level prediction toward read performance of high-density flash memories," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 10, pp. 3264–3274, Oct. 2023.
- [3] S. Pang, Y. Deng, Z. Wu, G. Zhang, J. Li, and X. Qin, "RDA: A read-request driven adaptive allocation scheme for improving SSD performance," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 2, pp. 416–429, Feb. 2025.
- [4] R. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, Jan. 1962.

- [5] Y. Zhang, Q. Cao, S. Wang, J. Yao, and H. Jiang, "HF-LDPC: HLS-friendly QC-LDPC FPGA decoder with high throughput and flexibility," in *Proc. IEEE 41st Int. Conf. Comput. Design (ICCD)*, 2023, pp. 566–573.
- [6] M. Zhang, F. Wu, Q. Yu, W. Liu, Y. Wang, and C. Xie, "Exploiting error characteristic to optimize read voltage for 3-D NAND flash memory," *IEEE Trans. Electron Devices*, vol. 67, no. 12, pp. 5490–5496, Dec. 2020.
- [7] Y. Song, Y. Lv, and L. Shi, "DECC: Differential ECC for read performance optimization on high-density NAND flash memory," in *Proc. 28th Asia South Pac. Design Autom. Conf. (ASP-DAC)*, 2023, pp. 1–6.
- [8] F. Amirzade, M.-R. Sadeghi, and D. Panario, "Construction of protograph-based LDPC codes with chordless short cycles," *IEEE Trans. Inf. Theory*, vol. 70, no. 1, pp. 51–74, Jan. 2024.
- [9] Q. Wang, S. Cai, and X. Ma, "Free-ride coding for constructions of coupled LDPC codes," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1259–1270, Mar. 2023.
- [10] S. Wang et al., "Lifetime adaptive ECC in NAND flash page management," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2017, pp. 1253–1556.
- [11] C. Zambelli et al., "Characterization of TLC 3D-NAND flash endurance through machine learning for LDPC code rate optimization," in *Proc. IEEE Int. Memory Workshop (IMW)*, 2017, pp. 1–4.
- [12] F. Wu et al., "Using error modes aware LDPC to improve decoding performance of 3-D TLC NAND flash," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 4, pp. 909–921, Apr. 2020.
- [13] Q. Xiong et al., "Characterizing 3D floating gate NAND flash: Observations, analyses, and implications," *ACM Trans. Storage*, vol. 14, no. 2, pp. 1–31, 2018.
- [14] Y. Li, G. Han, S. Huang, C. Liu, M. Zhang, and F. Wu, "Exploiting Metadata to estimate read reference voltage for 3-D NAND flash memory," *IEEE Trans. Consum. Electron.*, vol. 69, no. 1, pp. 9–17, Feb. 2023.
- [15] K.-K. Yong and L.-P. Chang, "Error diluting: Exploiting 3-D NAND flash process variation for efficient read on LDPC-based SSDs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 11, pp. 3467–3478, Nov. 2020.
- [16] W. Liu et al., "Characterizing the reliability and threshold voltage shifting of 3D charge trap NAND flash," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2019, pp. 312–315.
- [17] L. Cui, F. Wu, X. Liu, M. Zhang, and C. Xie, "VaLLR: Threshold voltage distribution aware LLR optimization to improve LDPC decoding performance for 3D TLC NAND flash," in *Proc. IEEE 37th Int. Conf. Comput. Design (ICCD)*, 2019, pp. 668–671.
- [18] Y. Cai, Y. Luo, E. F. Haratsch, K. Mai, and O. Mutlu, "Data retention in MLC NAND flash memory: Characterization, optimization, and recovery," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2015, pp. 551–563.
- [19] X. Liu, X. Wu, and C. Zhao, "Shortening for irregular QC-LDPC codes," *IEEE Commun. Lett.*, vol. 13, no. 8, pp. 612–614, Aug. 2009.
- [20] S. Mhaske, H. Kee, T. Ly, A. Aziz, and P. Spasojevic, "High-throughput FPGA-based QC-LDPC decoder architecture," in *Proc. IEEE 82nd Veh. Technol. Conf. (VTC)*, 2015, pp. 1–5.
- [21] L. Cui, F. Wu, X. Liu, M. Zhang, R. Xiao, and C. Xie, "Improving LDPC decoding performance for 3D TLC NAND flash by LLR optimization scheme for hard and soft decision," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 27, no. 1, pp. 1–20, Sep. 2021.
- [22] G. Yang et al., "High-precision error bit prediction for 3D QLC NAND flash memory: Observations, analysis, and modeling," *IEEE Trans. Comput.*, vol. 74, no. 4, pp. 1392–1404, Apr. 2025.



Lanlan Cui (Member, IEEE) received the Ph.D. degree from Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan, China, in 2021.

She is currently a Teacher with the School of Computer Science and Engineering, Xi'an University of Technology, Xi'an, China. Her current research interests include error correction codes, flash memory reliability, and nonvolatile storage.



Fei Wu (Member, IEEE) received the B.S. and M.S. degrees in electrical automation from the Wuhan University of Technology, Wuhan, China, in 1997 and 2000, respectively, and the Ph.D. degree in computer science from the Huazhong University of Science and Technology (HUST), Wuhan, in 2005.

She is currently a Professor with the Information Storage Lab, Wuhan National Laboratory for Optoelectronics, HUST. Her research interests include the computer architecture, nonvolatile memory, embedded systems, and intelligent storage.



Xuan Zhang received the B.S. degree in computer science from Xi'an University of Technology, Xi'an, China, in 2003, and the M.S. and Ph.D. degrees in computer science from Xidian University, Xi'an, in 2009 and 2019, respectively.

He is currently with the School of Computer Science and Engineering, Xi'an University of Technology. His major research interests include error-correcting codes and decoding algorithms.



Meng Zhang received the Ph.D. degree in computer science and technology from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2019.

He is currently an Associate Professor with the School of Computer Science and Technology, HUST. His current research interests include non-volatile memory, data storage reliability, and error correction codes.



Kun Jiang received the B.S., M.S., and Ph.D. degrees in computer science and technology from the College of Computer, National University of Defense Technology, Changsha, China, in 2008, 2011, and 2015, respectively.

From 2016 to 2018, he held a postdoctoral research position with Xi'an Jiaotong University, Xi'an, China. He is currently working as a Lecturer with the School of Computer Science and Engineering, Xi'an University of Technology, Xi'an.

He has been authored and co-authored more than 30 research articles in the fields of big data processing and artificial intelligence. His current research interest includes feature processing, representation learning, and optimization theory.



Zhazhan Zhao received the B.S. and Ph.D. degrees from Xidian University, Xi'an, China, in 2016 and 2022, respectively.

She is currently with the School of Computer Science and Engineering, Xi'an University of Technology, Xi'an. Her main research interests include error-correcting coding, information theory, and optimization methods for error correcting codes.



Changsheng Xie (Member, IEEE) received the B.S. and M.S. degrees in computer science and technology from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 1982 and 1988, respectively.

He is currently a Professor with the Wuhan National Laboratory for Optoelectronics, HUST, where he served as the Deputy Director of the Wuhan National Laboratory for Optoelectronics. His research interests include new storage technology and architecture, multimedia computing and network, computer storage, and network storage and security.

Mr. Xie served as the Deputy Director Computer Peripheral Equipment Committee of China Computer Federation, the Committee Member of the Information Storage of China Computer Society, and the Vice Chairman of the China Expert Committee of the International Network Storage Industry Association.